Hybrid LSTM-Transformer for RONIN Heading Estimation

Field	Value
Model Name	Hybrid LSTM-Transformer
Contributor	Gopal Datt Bhatt

1. Architecture

1.1 Model Description

This model estimates walking heading from RONIN smartphone sensor data using a Hybrid LSTM-Transformer architecture. Each input is a one-second sensor window with shape (200, 9), and the model predicts sin(heading) and cos(heading) instead of heading directly in degrees. The architecture combines an LSTM front-end to capture local temporal motion patterns with a Transformer encoder to model broader contextual relationships across the full sequence window.

1.2 Problem Setup and Input Representation

The model receives sliding windows constructed from accelerometer, gyroscope, and magnetometer channels. These three sensor modalities are fused into a multivariate input sequence of shape (200, 9), which is used as one model input sample. Predicting sine and cosine of heading is appropriate because heading is a circular variable, and representing the target in sin/cos space avoids the discontinuity between 0° and 360°.

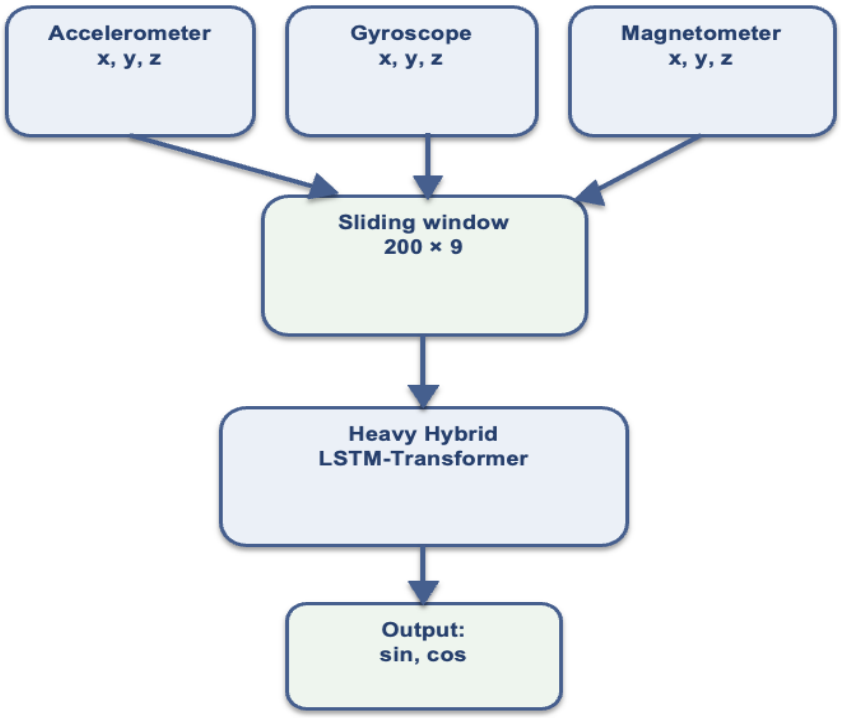


Figure 1. Input representation and prediction pipeline

1.3 Layer Structure

Component	Details
Input	200 × 9 sensor window
LSTM section	2-layer LSTM, hidden size 128, dropout 0.3
Transition	Full LSTM output sequence is linearly projected before Transformer encoding
Transformer section	2 encoder layers, 4 attention heads, model dimension 128, feedforward dimension 256, positional encoding used
Pooling	Mean pooling applied to Transformer output
Fusion	Final LSTM hidden state concatenated with mean-pooled Transformer summary
Fully connected output	256 → 64 → 2 output neurons

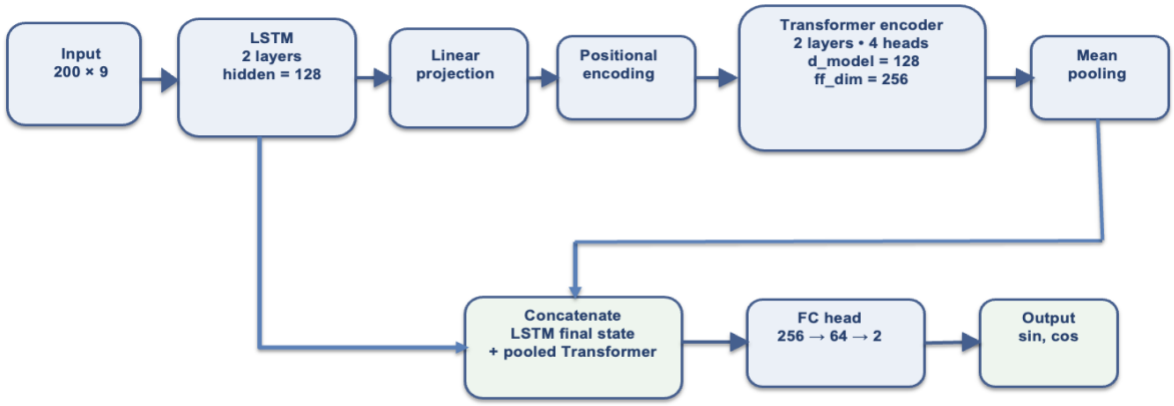


Figure 2. Hybrid LSTM-Transformer architecture.

The architecture first extracts temporal features using a stacked LSTM, then refines sequence-level context through a Transformer encoder. The final prediction is made from a fused representation combining the final LSTM hidden state and the pooled Transformer output.

1.4 Trainable Parameters

Total trainable parameters: 501,314.

1.5 Architectural Choices and Justification

A hybrid architecture was selected because its two branches serve different but complementary functions. The LSTM branch is well suited to modelling ordered temporal dynamics in smartphone motion sequences, while the Transformer branch can capture broader contextual relationships by relating distant positions within the same sensor window through self-attention. Using a hidden size of 128 and two LSTM layers gives the recurrent branch sufficient capacity to learn motion patterns without making it excessively shallow. The Transformer configuration, with model dimension 128, four attention heads, and two encoder layers, provides a richer contextual representation across the full input window. Mean pooling is used to compress the Transformer output into a sequence-level summary, and concatenating this summary with the final LSTM hidden state creates a fused feature vector containing both sequential and contextual information before the final prediction stage.

2. Training Setup

2.1 Loss Function

Mean Squared Error (MSE) loss was used on the two outputs [sin(heading), cos(heading)]. This works well for circular heading prediction because it avoids the discontinuity between values such as 0° and 360°. During evaluation, the predicted sine/cosine pairs were converted back to angles in degrees and assessed with the circular error formula.

2.2 Training Hyperparameters

Hyperparameter	Value
Optimizer	Adam
Learning rate	0.001
Batch size	128
Number of epochs	20 (maximum); early stopping triggered after epoch 11
Device used	CPU

2.3 Learning Rate Scheduling

Yes. **ReduceLROnPlateau** was used on validation loss with factor 0.5, patience 3, and minimum learning rate 1e-6. The learning rate stayed at 0.001 for epochs 1 to 6 and dropped to 0.0005 from epoch 7 onward when validation performance stopped improving.

2.4 Early Stopping and Timing

Early stopping was enabled so that training could terminate when further validation improvement became unlikely. The executed run recorded a total training time of 20,365.00 seconds, an average validation time of 168.10 seconds per epoch, and a final test inference time of 134.65 seconds. Although the maximum training limit was 20 epochs, training stopped after epoch 11 because validation loss did not improve enough after the earlier best epoch.

2.5 Training Difficulties

The main difficulty was computational cost. The hybrid model combines a recurrent branch and an attention-based branch, and the training run was carried out on CPU. This made each epoch relatively slow. A second challenge was maintaining generalisation on unseen data. The training and validation losses do not behave identically, so timing-aware monitoring, scheduling, and early stopping were important parts of the final pipeline.

3. Results

3.1 Quantitative Results

Metric	Value
MAE on test set (degrees)	66.1114
RMSE on test set (degrees)	82.8781
Approximate training time	20,365.00 s
Average validation time per epoch	168.10 s
Test inference time	134.65 s

3.2 Training Strategy and Loss Curve

The x-axis shows epoch number, the y-axis shows the loss value, and the two curves correspond to training and validation behaviour. The figure is useful because it shows how optimisation progressed over time and whether validation performance remained aligned with training performance.

Figure 3. Training strategy and loss curve.

3.3 Predicted vs Ground Truth Trajectory

The next figure presents one full unseen test trajectory, sequence a006_2, containing 3,475 window-level predictions. The two lines represent predicted heading and ground-truth heading over the complete sequence. This is more informative than plotting a short sample because it shows sequence-level behaviour across the whole trajectory.

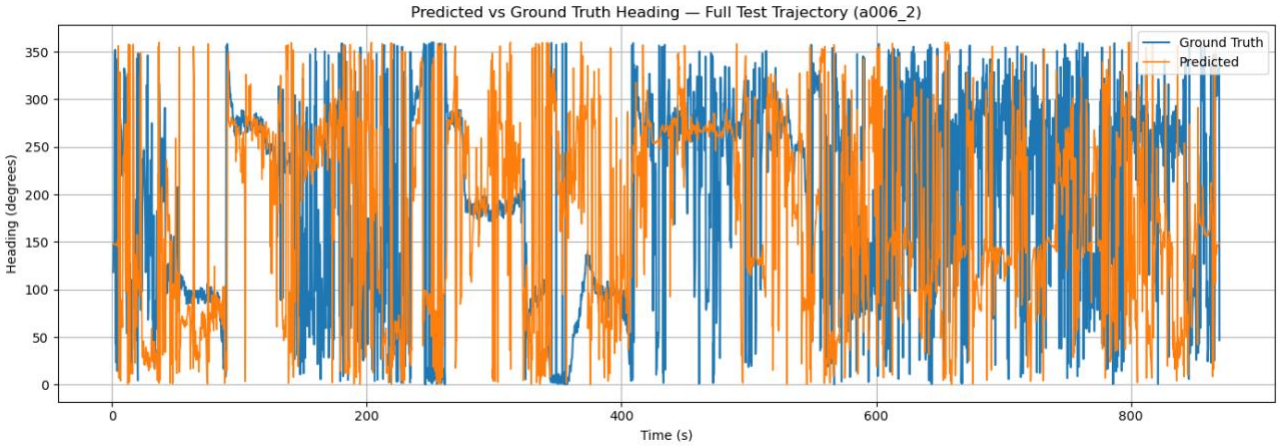
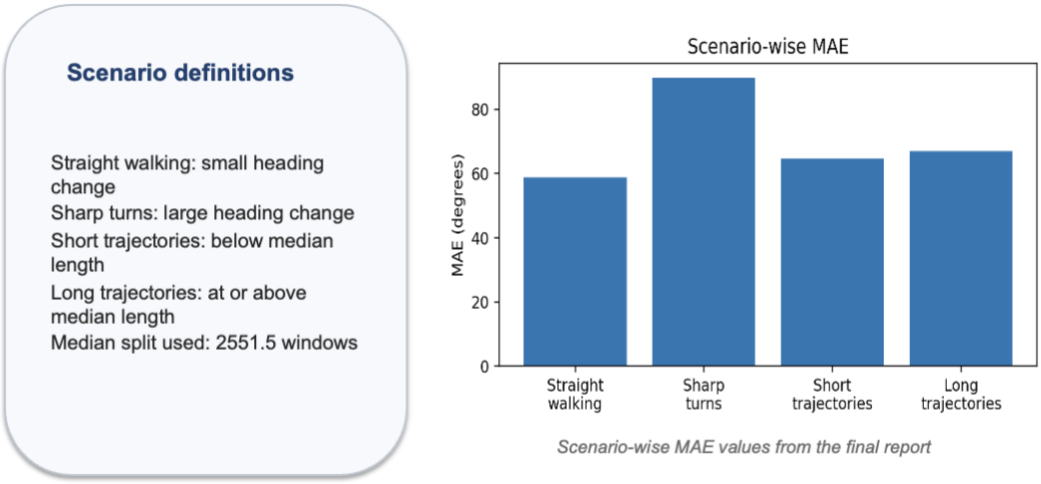


Figure 4. Full-trajectory heading plot

3.4 MAE Breakdown by Scenario

Scenario	MAE (degrees)
Straight walking	58.7827
Sharp turns	89.8205
Short trajectories	64.8144
Long trajectories	66.9763

Scenario-wise MAE was computed using circular heading error. Straight walking was defined as windows with ground-truth heading change <= 10°, while sharp turns were defined as windows with heading change >= 45°. Short and long trajectories were defined using a median split on the number of windows per unseen test trajectory. The median trajectory length was 2551.5 windows, so trajectories with 2551 windows or fewer were treated as short and trajectories with 2552 windows or more were treated as long.



.

Figure 5. Scenario-wise MAE analysis

4. Observations and Technical Reflection

4.1 Where the Model Performs Well

The model performs comparatively better when movement is smoother and heading changes are more stable. This is reflected in the lowest scenario-wise MAE being obtained for straight walking. In such situations, the sequential structure captured by the LSTM and the wider contextual summary captured by the Transformer appear to support more reliable heading tracking.

4.2 Where the Model Struggles

The most difficult case is sharp turns, which produced the highest scenario-wise MAE. Abrupt directional changes are harder to model accurately because they involve rapid transitions in heading over short time spans. The full-trajectory figure also shows regions where predicted and ground-truth headings diverge substantially, indicating difficulty under highly variable motion.

4.3 Loss Curve Interpretation

The loss-curve figure shows that the model learns meaningful structure during training: the training loss decreases steadily. Validation behaviour remains informative but does not improve at the same rate, which suggests that the model fits the training data more easily than it generalises to unseen sequences. This makes the loss curve useful evidence that optimisation is successful, but generalisation remains the main challenge.

4.4 Architecture vs Training — Root Cause of Limitations

The observed limitations likely come from both model design and training constraints. The hybrid architecture is expressive enough to learn the training signal, but its error profile on unseen test data shows that hard cases remain unresolved. Part of the limitation is architectural: although the model fuses LSTM and Transformer information, a

pooled summary can still lose some fine-grained heading dynamics. Part of the limitation is also computational: CPU training limited how extensively the heavy configuration could be tuned and repeated.

4.5 One Change I Would Make

If more time and compute were available, the next improvement would be stronger systematic tuning of the fusion stage and sequence summarisation while training on GPU. This would make it practical to test whether better fusion strategies, more runs, and more aggressive hyperparameter search can reduce the large error observed on sharp-turn scenarios.

5. Summary

Overall, the Hybrid LSTM-Transformer is a complete evaluation-ready heading-estimation pipeline. It integrates an LSTM branch and a Transformer branch, predicts heading in sine-cosine form, records timing information, and provides both aggregate and scenario-wise performance analysis.